

COL216 Assignment 3 - Semester 2024-20245 Sem II

Eshita Zjigyasu (2023CS51215) — Srushti Pazare (2023CS11000)

April 30, 2025

1 Implementation

1.1 Data Structures Used

We have used data structures such as classes, structs, enums and vectors. The following are the main classes that we have in our simulator:

1. CacheLine
2. Set
3. Cache
4. Core
5. Bus
6. Processor

The following are the main structs that we have in our simulator:

1. BusMessage - contains all the information for setting the time for which bus needs to be free, and what state changes to do when the bus gets free
2. Instruction - contains a boolean value for isRead, and an integer value for address
3. RunResult - contains the result of the run() function of a core (the information about current instruction, whether it was a hit or miss, which line to evict, etc.)

Each processor has a vector pointers to cores, and a vector of pointers to caches. It also has a pointer to an object of the Bus class. The Processor constructor creates these cores, caches and the bus on the dynamic memory, and we are ensuring that this memory is freed when it is destructed, hence ensuring there are no memory leaks.

For each core, we have modeled the trace as a vector of Instruction objects. It also has attributes like number of reads, number of writes, etc. for calculating the statistics at the end. The Core class has a function run(), which is called when the core gets unblocked. This function takes the next instruction, and accesses the cache.

Each cache is modeled as a vector of pointers to Set objects. This is done by making an attribute **sets** of type `std::vector<Set*>`. Similarly, a Set object has an attribute **lines** of type `std::vector<CacheLine*>`

The Bus object holds an attribute **message** of type `BusMessage`, and has a function **requestBus**.

We have a few values set as globals, such as **cycleCount**, number of sets, number of blocks, and block size. We have set the default as a 4KB 2-way set associative L1 cache per processor, with 32-byte block size. These are the values that will be considered if the simulator is run without the appropriate arguments. If arguments are given, these default values will be overwritten.

1.2 Flow of the program

In the main function, we create an object of Processor class, and give it the traces that we have read from the appropriate trace files given as command line input.

Until all the cores have completed all of their instructions, we call the `processor.simulate()` function, which simulates one clock cycle.

Inside the simulate function, we do the following.

1. First check if the bus is performing some transaction or not. If the bus is busy, we check if it is supposed to get free in this cycle, and then make the state changes considering the operation on the bus has been finished successfully.

2. Then we iterate on all the cores one by one, in the order core0, core1, core2 and core3. If a core is blocked, we check if it's getting free in this cycle, and then make the appropriate changes in the cache. If a core is waiting for the bus, then we request the bus once again.
3. If a core is free, we call the `run()` method on it. We process the result of the run method and accordingly make a bus request, or wait for the bus until it gets free.
4. After the bus request has been made, we have the algorithm for snooping. Each cache sees the message on the bus, and checks if it has the corresponding address on its cache. According to the MESI protocol, it then either does nothing, or changes the message on the bus to do a cache to cache transfer.

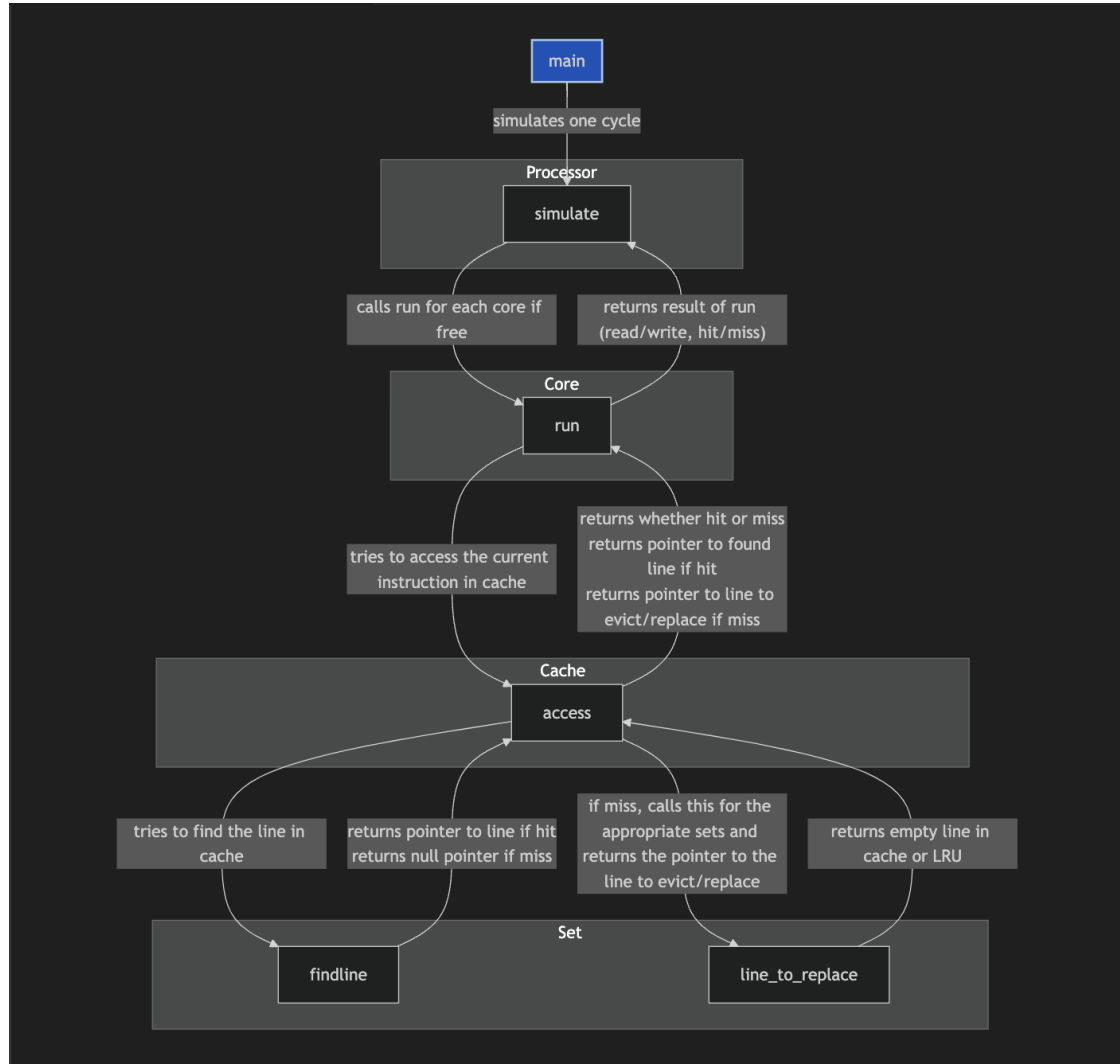


Figure 1: Flowchart

2 Distributions of Outputs

Since our program is deterministic, there is no variance in the outputs for different runs. We are always giving priority to the cache with a lower number, instead of choosing a cache arbitrarily.

3 Observations

We get this graph because as the number of sets increases, we will get more hits.

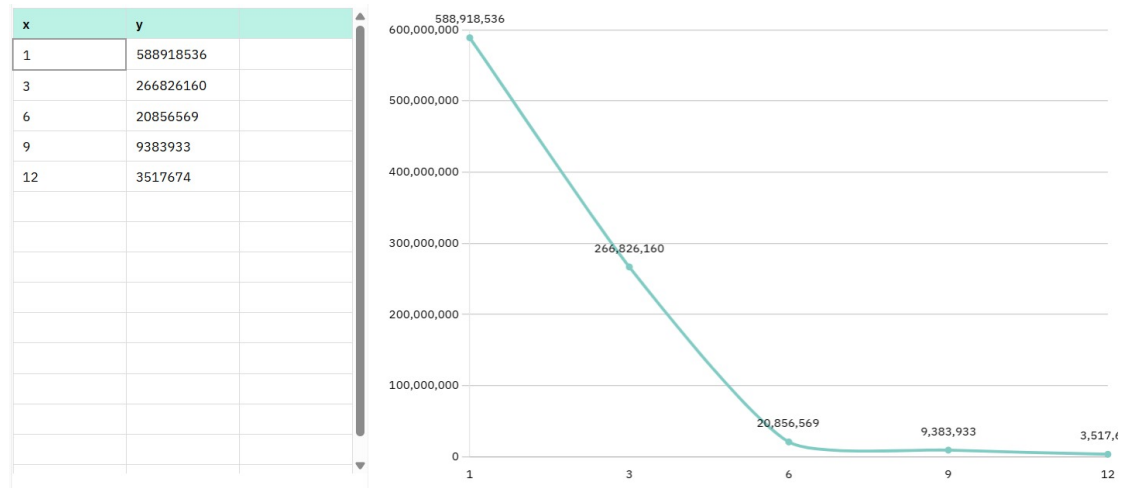


Figure 2: Varying the number of sets, plotting with max number of cycles
We get this graph because as the number of sets increases, we will get more hits.

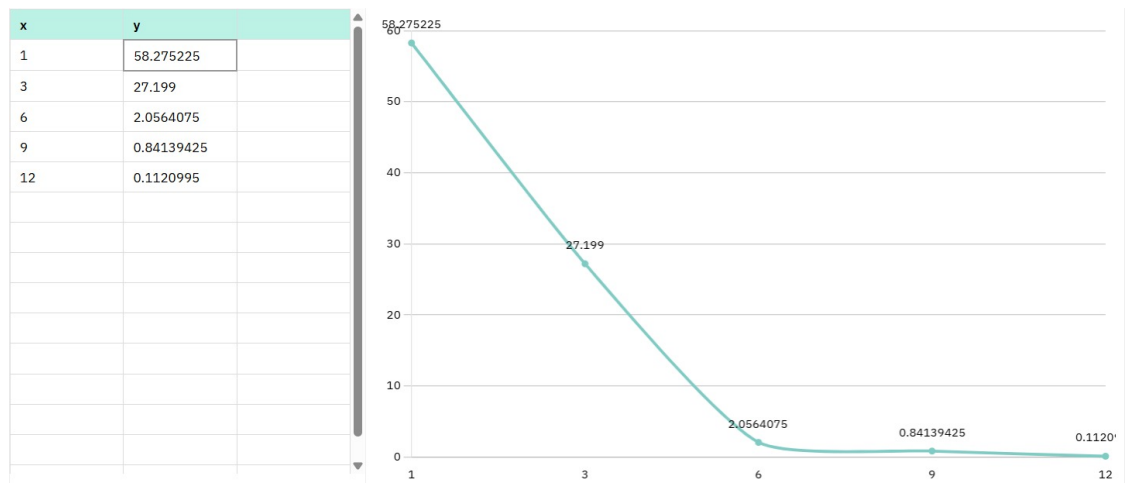


Figure 3: Varying the number of sets, plotting with average miss rate

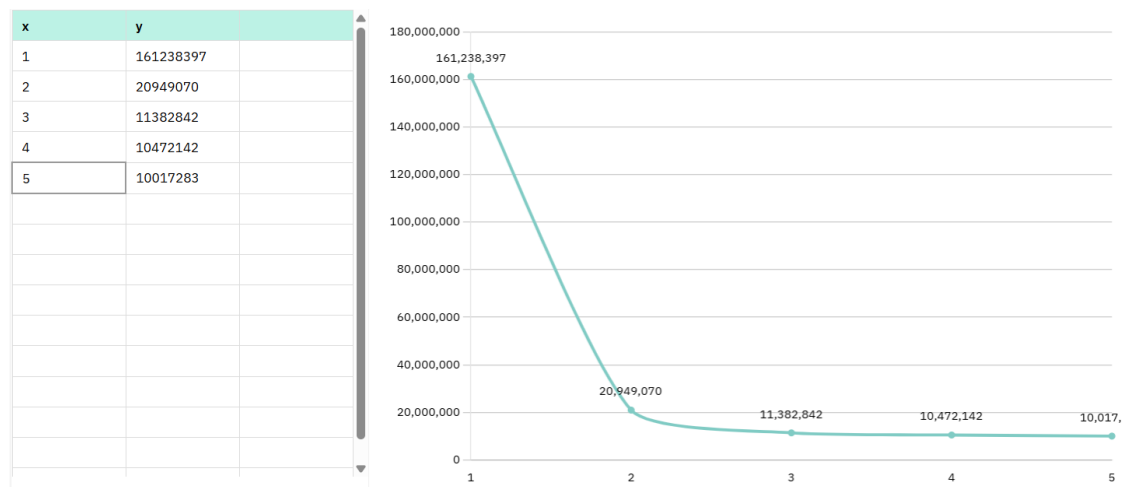


Figure 4: Varying the number of blocks per set, plotting with max number of cycles
Increasing the number of blocks per set decreases the number of cycles due to more hits.

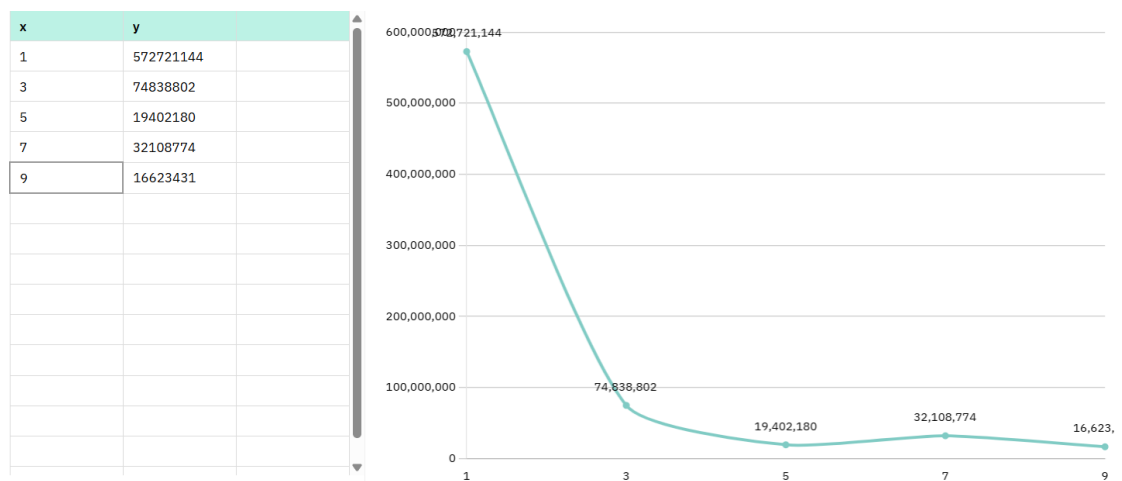


Figure 5: Varying the block size, plotting with max number of cycles
Increasing the number of block size decreases the number of cycles due to more hits.